COLLEGE OF COMPUTING AND DATA SCIENCE
NANYANG TECHNOLOGICAL UNIVERSITY

SC4003: INTELLIGENT AGENTS

Assignment 1 Report

Leonard Ong Kai Jun
U2222385H

Contents

1 Introduction 1
1.1 Part 1 1
1.2 Part 2 2
2 Project Structure 3
2.1 `config.py` – Shared Constants 3
2.2 `maze.py` – Maze Environment 3
2.3 `value_iteration.py` – Value Iteration 4
2.4 `policy_iteration.py` – Policy Iteration 4
2.5 `utils.py` – Printing & Plotting 4
2.6 `main.py` – Entry Point 4
2.7 `maze_configs/` – Maze Definitions 4
3 Part 1: Base 6 × 6 Maze 5
3.1 Maze Layout 5
3.2 Value Iteration 5
3.2.1 Description of Implementation 5
3.2.2 Optimal Policy 6
3.2.3 Utilities of All States 7
3.2.4 Plot of Utility Estimates as a Function of Iterations (Value Iteration) 9
3.3 Policy Iteration 10
3.3.1 Description of Implementation 10
3.3.2 Optimal Policy 11
3.3.3 Utilities of All States 12
3.3.4 Plot of Utility Estimates as a Function of Iterations (Policy Iteration) 13
4 Part 2: More Complex Maze Environments 15
4.1 Maze 1: Increased-Size (10×10, 86 states, 14 walls) 15
4.2 Maze 2: Labyrinth (10×10, 67 states, 33 walls) 18
4.3 Maze 3: Minefield (10×10, 90 states, 10 walls) 20
4.4 Analysis: Effect of Complexity on Convergence 22
4.4.1 How does the number of states and the complexity of the environment affect convergence? 22
4.4.2 How complex can you make the environment and still be able to learn the right policy? 23
5 Conclusion 24

Chapter 1

Introduction

The assignment considers a 6× 6 stochastic maze environment (as stated in Assignment 1 of SC4003) with the following properties:

- **Transition model**: the intended action succeeds with probability 0.8;

with probability 0.1 the agent moves at a right angle to the intended direction (0.1 each side). If the move would make the agent walk into a wall, the agent stays in the same place as before.

- **Rewards**: green cells yield $+1$, brown cells yield -1, and white (empty) cells yield -0.05.
- **No terminal states**: the agent's state sequence is infinite.

Figure 1: Base 6×6 maze from the assignment. Green = +1, Brown = −1, White = −0.05.

Two tasks are posed:

1.1 Part 1

Assuming the known transition model and reward function listed above, find the optimal policy and the utilities of all the (non-wall) states using both value iteration and policy iteration. Display the optimal policy and the utilities of all the states, and plot utility estimates as a function of the number of iterations as in Figure 17.5(a) in the reference book. Use a discount factor of $\gamma = 0.99$.

1.2 Part 2

Design a more complicated maze environment of your own and re-run the algorithms designed for Part 1 on it. How does the number of states and the complexity of the environment affect convergence? How complex can you make the environment and still be able to learn the right policy?

Chapter 2

Project Structure

The codebase is organised as follows:

```
/
├── config.py                # Shared constants & configuration
├── maze.py                  # Maze environment class
├── value_iteration.py       # Value Iteration algorithm
├── policy_iteration.py      # Policy Iteration algorithm
├── utils.py                 # Printing & plotting utilities
├── main.py                  # Entry point — runs all experiments
├── results/
└── maze_configs/            # Maze definitions (Part 1 & Part 2)
  ├── init.py
  ├── base.py              # Part 1: base 6×6 maze
  ├── increased_size.py    # Part 2: 10×10 open maze
  ├── labyrinth.py         # Part 2: 10×10 narrow corridors
  └── mine_field.py        # Part 2: 10×10 scattered -1 penalties
```

Listing 1: Project file structure.

Each module plays a specific role in the pipeline:

2.1 `config.py` – Shared Constants

Defines the discount factor ($\gamma = 0.99$), the convergence tolerance derived from the AIMA formula ($\delta < \varepsilon \frac{1-\gamma}{\gamma} \approx 0.00101$ with $\varepsilon = 0.1$), the four cardinal actions (N, S, E, W) and their deltas, the stochastic transition probabilities (0.8 intended, 0.1 each perpendicular), and the perpendicular direction mapping for each action.

2.2 `maze.py` – Maze Environment

Implements the `Maze` class which represents the grid world. It stores the grid dimensions, wall positions, and reward values for every non-wall cell. Key methods include:

- `is_valid(r, c)`: checks whether a cell is in-bounds and not a wall.
- `next_state(state, action)`: returns the resulting state after attempting an action;

if the move would hit a wall or boundary, the agent stays put.

- `transition_probs(state, action)`: computes the full probability distribution $P(s' \mid s, a)$ over successor states for a given action, correctly accumulating probabilities when multiple stochastic outcomes map to the same cell (e.g., two perpendicular moves both bouncing off a wall).
- `expected_value(state, action, utility)`: computes $\sum_{s'} P(s' \mid s, a) \cdot U(s')$.

2.3 `value_iteration.py` – Value Iteration

Implements the Bellman update loop: starting from $U_0(s) = 0$ for all states, it iteratively applies $U_{i+1}(s) = R(s) + \gamma \cdot \max_a \sum_{s'} P(s'|s, a) \cdot U_{i(s')}$ until $\delta <$ tolerance. After convergence, the optimal policy is extracted greedily. Optionally tracks one state's utility history for plotting.

2.4 `policy_iteration.py` – Policy Iteration

Implements the two-step loop. Policy evaluation solves the linear system $(I - \gamma T_\pi)U = R$ exactly using `numpy.linalg.solve`. Policy improvement then greedily updates each state's action. The loop terminates when no state changes its action. The initial policy is randomly assigned.

2.5 `utils.py` – Printing & Plotting

Provides functions for terminal output (utility grids and arrow-based policy grids) and for generating matplotlib convergence plots — both single-tracked-state curves and all-states-on-one-figure plots. Also contains `value_iteration_all_states` and `policy_iteration_all_states`, which are wrappers that record every state's utility at each iteration for the all-states convergence figures.

2.6 `main.py` – Entry Point

Orchestrates the full experiment: builds each maze, runs both VI and PI, prints results to the terminal, and saves convergence plots as `.png` files in the `results/` directory.

2.7 `maze_configs/` – Maze Definitions

Each file (e.g., `base.py`, `increased_size.py`, `labyrinth.py`, `mine_field.py`) defines a maze by specifying the grid dimensions, wall positions, green (+1) and brown (−1) reward cells, start position, and a `build()` function that constructs and returns a `Maze` instance.

Chapter 3

Part 1: Base 6 × 6 Maze

Task: *Assuming the known transition model and reward function, find the optimal policy and the utilities of all (non-wall) states using both value iteration and policy iteration. Display the optimal policy and the utilities of all states, and plot utility estimates as a function of the number of iterations. Use* $\gamma = 0.99$.

3.1 Maze Layout

The maze is a 6 × 6 grid with 5 wall cells and 31 non-wall (traversable) states:

Table 1: Base 6×6 maze layout. Green = +1, Brown = −1, White = −0.05. Start at (3, 2).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 0	**+1**	Wall	**+1**	−0.05	−0.05	**+1**
Row 1	−0.05	**−1**	−0.05	**+1**	Wall	**−1**
Row 2	−0.05	−0.05	**−1**	−0.05	**+1**	−0.05
Row 3	−0.05	−0.05	Start	**−1**	−0.05	**+1**
Row 4	−0.05	Wall	Wall	Wall	**−1**	−0.05
Row 5	−0.05	−0.05	−0.05	−0.05	−0.05	−0.05

3.2 Value Iteration

3.2.1 Description of Implementation

Value Iteration applies the Bellman equation iteratively to find the optimal utilities of all states. For each state s, the utility is updated using the following formula:

$$U_{i+1}(s) = R(s) + \gamma \cdot \max_a \sum_{s'} P(s' \mid s, a) \cdot U_i(s')$$

Where:

- $U_{i+1}(s)$ is the updated utility of state s at iteration $i + 1$.
- $R(s)$ is the immediate reward received in state s.
- γ is the discount factor, which determines the present value of future rewards.

- $\max_a$ represents the agent choosing the action a that maximises expected future utility.
- $P(s' \mid s, a)$ is the transition model, representing the probability of reaching next state s' given action a in state s.
- $U_{i(s')}$ is the utility of the next state s' from the current iteration i.

Starting from an initial utility of $U_0(s) = 0$ for all states, the algorithm continuously sweeps through the state space updating these values. It terminates based on the convergence criterion defined in the Artificial Intelligence: A Modern Approach (AIMA) reference textbook, which guarantees that the resulting policy is within a specific error margin of the true optimal policy.

Specifically, the algorithm halts when the maximum utility change across all states during a single sweep, denoted as δ, satisfies:

$$\delta < \frac{\varepsilon(1-\gamma)}{\gamma}$$

For this implementation, the error tolerance parameter is set to $\varepsilon = 0.1$ and the discount factor is $\gamma = 0.99$. Substituting these values into the criterion yields a strict stopping condition of $\delta < 0.00101$.

3.2.2 Optimal Policy

The algorithm converged in **688 iterations**:

Table 2: Optimal policy from Value Iteration (▦ = wall).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 0	↑	▦	←	←	←	↑
Row 1	↑	←	←	←	▦	↑
Row 2	↑	←	←	↑	←	←
Row 3	↑	←	←	↑	↑	↑
Row 4	↑	▦	▦	▦	↑	↑
Row 5	↑	←	←	←	↑	↑

Analysis:

The derived policy clearly illustrates the agent balancing the pursuit of positive rewards against the risk of falling into penalties (-1) due to the stochastic transition model. Because the discount factor is very high ($\gamma = 0.99$), the agent prioritises safety over speed, which mathematically incentivises to a longer, circuitous route rather

than risking a shorter path near a hazard where a 10% perpendicular drift could prove costly.

The state space is broadly divided into two main attractor basins:

- **The (0,0) Safe (Columns 1, 2, and 3) :** The upper-left quadrant strongly funnels towards the +1 reward at (0,0). This state has the highest overall utility because it is nestled against the grid boundaries and the wall at (0,1). This surrounding geometry acts as a physical buffer, where if the agent intends to move towards the wall, perpendicular stochastic drifts simply bounce it back into its current state, drastically reducing the risk of accidentally drifting into the −1 penalty at (1,1).
- **The Right-Side Integration (Columns 4,5):** States in the right half of the maze are drawn towards the +1 rewards at (0,5) and (3,5). However, navigating this side is risky due to the −1 states scattered at (1,5), and (4,4). The policy meticulously routes the agent upward and leftward around these hazards.

A prime example of the agent's extreme risk aversion can be seen in its routing around the massive wall barrier `[▓, ▓, ▓]` in Row 4. Rather than trying to navigate the tight, dangerous gaps on the right side of the maze, the policy dictates that agents in the centre of Row 5 `(5,1 to 5,3)` must travel all the way left to Column 0. This creates a long, safe vertical corridor where the agent can travel cleanly upward, bypassing the wall barrier entirely without risking a stochastic drift into a penalty.

3.2.3 Utilities of All States

Table 3: Converged utilities from Value Iteration ($\gamma = 0.99$).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 0	99.90	▓	94.92	93.74	92.51	93.18
Row 1	98.28	95.77	94.42	94.27	▓	90.77
Row 2	96.82	95.45	93.16	93.03	92.96	91.64
Row 3	95.41	94.30	93.07	90.97	91.66	91.74
Row 4	94.16	▓	▓	▓	89.39	90.40
Row 5	92.77	91.55	90.35	89.16	88.40	89.12

The converged utilities form a distinct topographical gradient that reflects the infinite horizon, highly discounted nature of the environment ($\gamma = 0.99$). Because the discount factor is so close to 1, the long-term accumulation of future positive rewards vastly outweighs immediate penalties. This results in every traversable state possessing a highly positive expected utility, ranging roughly from 88 to 100.

Analysis:

State (0,0) emerges as the absolute peak of this topographical gradient, achieving the highest utility in the grid at 99.90. Because the optimal policy effectively turns (0,0) into a safe, absorbing state, which allows the agent to repeatedly bounce off the walls to collect the +1 reward indefinitely since its theoretical exact utility is $U = R/(1-\gamma) = 1.0/(1-0.99) = 100$. The recorded value of 99.90 simply reflects the algorithm halting prematurely once the maximum change δ drops below the established tolerance threshold. From this maximum, utilities smoothly degrade as the geometric distance from the primary (0,0) attractor increases. A clear example of this decay can be observed traveling down Column 0 from (0,0) to (5,0), where the utility drops sequentially from 99.90 down through 98.28, 96.82, 95.41, 94.16, and finally to 92.77. This gradual decline is a direct, spatial manifestation of the $\gamma = 0.99$ discount factor compounding over the discrete time steps required to travel upward to the goal.

Interestingly, even the −1 penalty states maintain highly positive values, with state (3,3) registering a depressed utility of 90.97 and state (4,4) sitting at 89.39. In the context of an infinite horizon MDP with such a high discount factor, a penalty is merely a temporary toll. The agent's policy will quickly manoeuvre it out of these states and navigate towards a +1 reward, meaning the utilities of these cells are only "depressed" relative to their safer neighbours rather than being inherently negative. In fact, the true global minimum of the grid is not a penalty state at all, but rather state (5,4), which holds the lowest utility at 88.40. This state is severely disadvantaged by its geography because it is isolated far from any +1 rewards, situated directly beneath the −1 penalty at (4,4), and completely cut off by the Row 4 wall barrier. This geographic isolation forces the agent into a long, highly discounted journey just to reach a safe haven, demonstrating that in this environment, a long path is far more punishing than a brief penalty.

3.2.4 Plot of Utility Estimates as a Function of Iterations (Value Iteration)

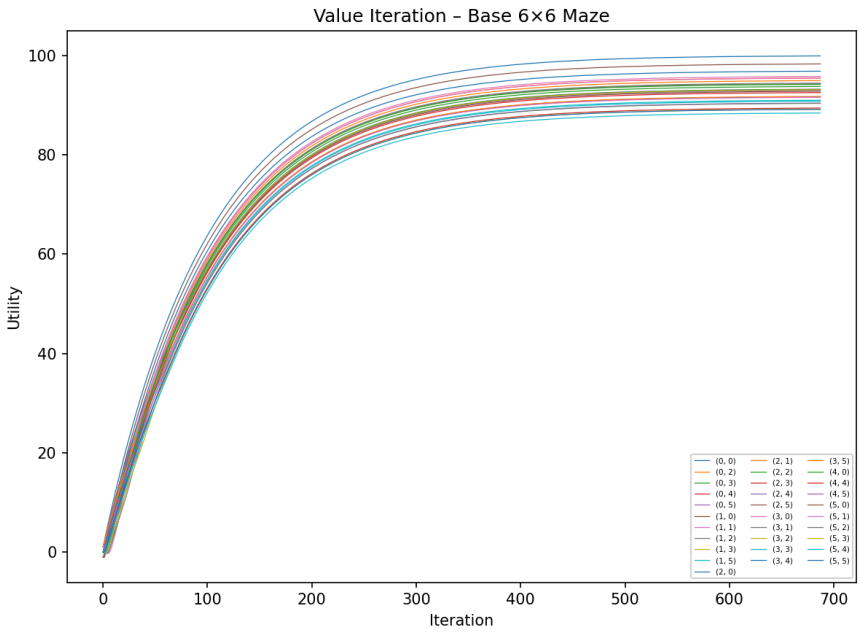

Figure 2: Utility estimates for all 31 states vs. iteration number (Value Iteration).

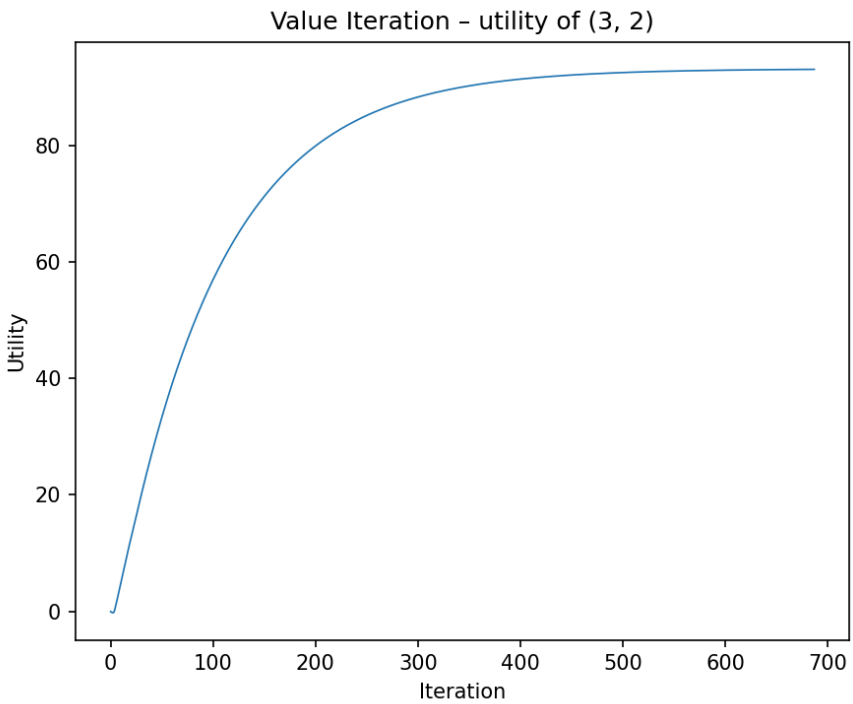

Figure 3: Convergence of the Start state (3, 2) (Value Iteration).

Both Figure 2 and Figure 3 illustrate the continuous, asymptotic convergence of state utilities over ~688 iterations. Because all states are initialised to an arbitrary $U_0(s) =$

0, the graph captures the exact step-by-step propagation of reward information across the grid.

Analysis:

In the first few iterations, only states that are immediately adjacent to the +1 and −1 cells will experience significant updates. Those distant states will remain near zero until the Bellman equation updates "ripple" outward across the grid. This effect can be seen from the tracked start state at (3,2) as seen from Figure 3. The start state is a prime example of a mid-tier state, where it begins at 0, experiences a slight delay before the "ripple" of positive rewards reach it, which then climbs smoothly to its final converged value of about 93 as the algorithm maps its optimal sequence of actions towards the distant goals. This step by step spatial propagation is the reason why Value Iteration takes hundreds of sweeps to solve the environment.

The curve rises rapidly at first, but the rate of change slows down drastically as it passes the 300 mark. This diminishing speed is a visual representation of the Bellman operator's contraction mapping property. Where each subsequent update adds a geometrically smaller fractional value ($\gamma = 0.99$) until the maximum change δ drops below 0.00101 AIMA tolerance limit.

As the algorithm progresses, the lines naturally group into bands based on their geographic advantages. The tightly clustered upper lines represent states in "safe havens", near the +1 rewards. While the lower lines belong to states that are isolated by walls or sit close to penalties.

3.3 Policy Iteration

3.3.1 Description of Implementation

Policy Iteration alternates between two steps:

1. **Policy Evaluation**: given a fixed policy π, solve the system of linear equations exactly:

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) \cdot U^{\pi}(s')$$

   This is reformulated as $(I - \gamma T_{\pi})U = R$ and solved via `numpy.linalg.solve`.
2. **Policy Improvement**: for each state, greedily select the action maximising expected utility:

$$\pi_{i+1}(s) = \arg\max_{a} \sum_{s'} P(s' \mid s, a) \cdot U^{\pi_i}(s')$$

If no state's action changes, the algorithm has converged.

The initial policy is randomly assigned.

3.3.2 Optimal Policy

The algorithm converged in **6–8 iterations** (varies with the random initial policy). The optimal policy is identical to that of Value Iteration:

Table 4: Optimal policy from Policy Iteration (identical to Value Iteration).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 0	↑	▦	←	←	←	↑
Row 1	↑	←	←	←	▦	↑
Row 2	↑	←	←	↑	←	←
Row 3	↑	←	←	↑	↑	↑
Row 4	↑	▦	▦	▦	↑	↑
Row 5	↑	←	←	←	↑	↑

Analysis:

As shown above in Table 4, Policy Iteration derives an optimal policy that is completely identical to the one produced by Value Iteration. This identical output serves as a strong mathematical validation of our implementation. While Value Iteration arrived at this routing through hundreds of incremental, asymptotic approximations (~688 iterations), Policy Iteration achieved the exact same result through a handful of exact linear evaluations and greedy improvements.

Because the underlying MDP parameters, specifically the transition model and the high $\gamma = 0.99$ discount factor, remain unchanged, the agent's calculated behavioural incentives are identical. The resulting policy reflects the exact same extreme risk aversion discussed previously, prioritising safe boundary corridors and actively routing away from the stochastic hazards near the -1 penalty states. The fact that two distinctly different mathematical approaches converged on the exact same navigational strategy confirms that this is the true, absolute optimal policy for this environment.

3.3.3 Utilities of All States

Table 5: Converged utilities from Policy Iteration ($\gamma = 0.99$).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 0	100.00	▦	95.02	93.84	92.61	93.28
Row 1	98.38	95.87	94.52	94.37	▦	90.87
Row 2	96.92	95.55	93.26	93.13	93.06	91.74
Row 3	95.51	94.40	93.17	91.07	91.76	91.84
Row 4	94.26	▦	▦	▦	89.49	90.50
Row 5	92.87	91.65	90.45	89.26	88.50	89.22

Policy Iteration yields the exact Bellman equation solution via linear algebra. The utilities are marginally higher than the VI approximation (max difference ~0.1, within the convergence tolerance). State (0,0) reaches $U = 100.00 = 1.0/(1 - 0.99)$, the theoretical maximum for a +1 reward state under an infinite-horizon discounted MDP with $\gamma = 0.99$.

Analysis:

Unlike the asymptotic approximation of Value Iteration, Policy Iteration leverages linear algebra during its evaluation step to solve the exact Bellman equations. As a result, the computed utilities represent the true mathematical limits of the environment rather than an approximation bound by a convergence tolerance. This exactness is perfectly demonstrated by state (0,0), which reaches its theoretical maximum utility of exactly 100.00 (calculated as $\frac{1.0}{1-0.99}$) under the infinite-horizon discounted MDP.

Overall, the utilities are marginally higher across the board (typically by about 0.1) compared to the Value Iteration results. This slight, uniform shift confirms that Value Iteration halted exactly as designed, where it is just shy of the absolute mathematical limit once the maximum utility change δ fell below the strict AIMA tolerance threshold. Despite this increase in numerical precision, the relative topographical gradient remains completely unchanged. The (0,0) state remains the absolute peak, and the isolated (5,4) state remains the global minimum (now precisely 88.50). Because the relative differences between adjacent states are preserved, the exact same optimal routing policy is extracted.

3.3.4 Plot of Utility Estimates as a Function of Iterations (Policy Iteration)

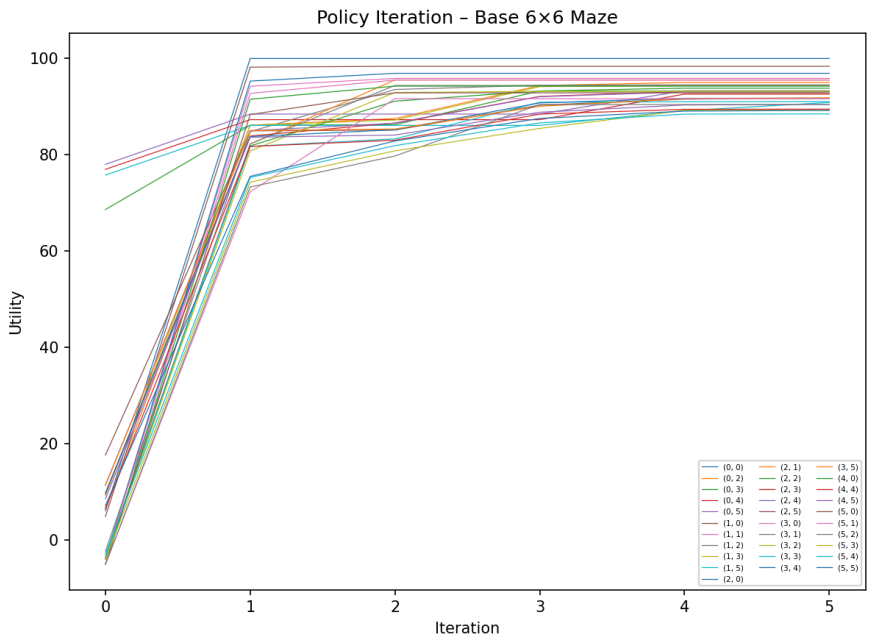

Figure 4: Utility estimates for all 31 states vs. iteration number (Policy Iteration).

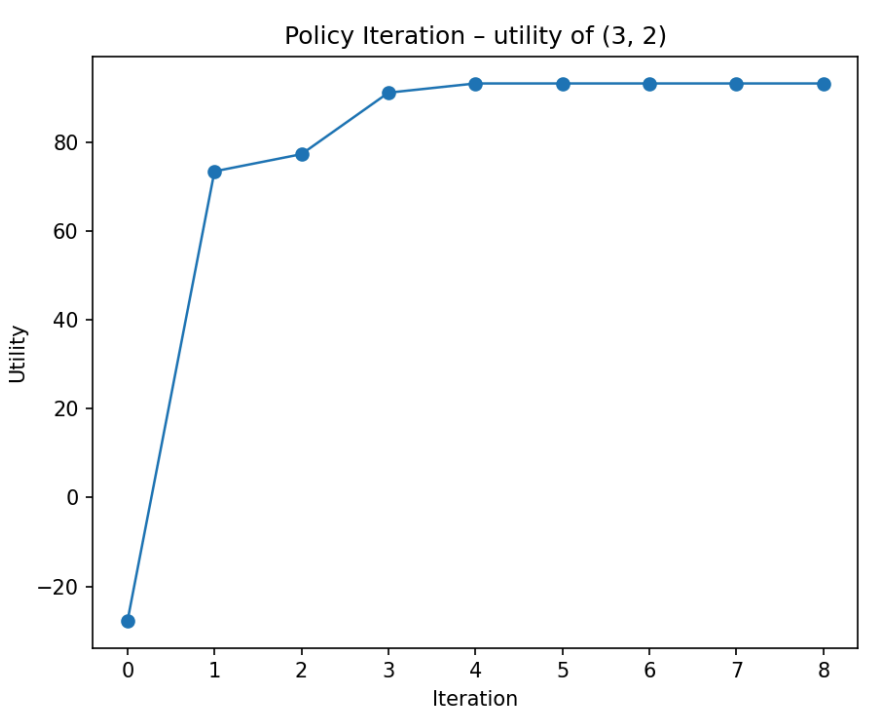

Figure 5: Convergence of the Start state (3, 2) (Policy Iteration).

Analysis:

The most striking contrast between the two algorithms lies in their convergence dynamics. As seen in Figure 4 and Figure 5, Policy Iteration solves the environment

in a mere handful of iterations, which is roughly two orders of magnitude fewer than Value Iteration's 688 sweeps. Because each step involves an exact policy evaluation that solves an $n \times n$ linear system, the state utilities do not slowly ripple outward. Rather, they immediately jump to their exact mathematical values for the current policy.

This aggressive correction is clearly visible in the tracking of the start state at (3,2). Because the initial policy is generated randomly, the exact starting utility fluctuates significantly across different executions. However, in the specific run captured in Figure 5, the state begins with an initial utility of roughly −28. Following the very first policy evaluation step, this value vaults dramatically to ~ 72. As the algorithm performs greedy policy improvements over the next few steps, the utility climbs to about 86 at Iteration 3, and perfectly flatlines at its converged optimal value of 93.17 by Iteration 4. Figure 4 reinforces this, showing a massive, nearly vertical unified jump at Iteration 1 across the entire grid. Ultimately, the algorithm requires only a few structural updates before the routing stabilises completely and exact convergence is achieved.

Chapter 4

Part 2: More Complex Maze Environments

To systematically investigate how state volume, structural density, and stochastic risk tolerance influence algorithm convergence, this chapter introduces three distinct 10×10 maze variations. All environments utilise the same transition model ($P_{\text{intended}} = 0.8$, $P_{\text{perp}} = 0.1$ each side), reward scheme (+1 green, −1 brown, −0.05 white), and discount factor ($\gamma = 0.99$). Rather than arbitrarily scaling the grid, each environment is deliberately engineered to isolate and test a specific dimension of MDP complexity.

- The **Increased-Size** maze serves as a baseline for spatial expansion, drastically increasing the raw number of traversable states while maintaining a relatively open layout to test pure volumetric scaling.
- To contrast this, the **Labyrinth** maze introduces severe structural density, testing how tightly constrained corridors and restricted action spaces impact the speed of policy updates.
- Finally, the **Minefield** maze evaluates the algorithms' handling of stochastic risk, forcing the agent to mathematically weigh the slow drain of step costs against the severe, probabilistic risk of slipping into a penalty trap.

By comparing the performance of Value Iteration and Policy Iteration across these three distinct geometric challenges, we can untangle whether algorithmic convergence is dictated by the sheer size of an environment or the physical complexity of its obstacles.

4.1 Maze 1: Increased-Size (10×10, 86 states, 14 walls)

This maze starts at (4,3) and scales the state count from 31 (in the base environment) to 86 while keeping the wall structure relatively open. Rewards and penalties are

scattered throughout the grid, testing how simply increasing the volume of states affects computational convergence.

Table 6: Increased-Size 10×10 maze layout. Green = +1, Brown = −1, White = −0.05. Start at (4, 3).

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	+1	−0.05	−0.05	−1	−1	−0.05	+1	+1	+1	+1
Row 1	−1	+1	−0.05	Wall	−0.05	−0.05	+1	+1	+1	−0.05
Row 2	Wall	−0.05	Wall	−0.05	−0.05	Wall	−0.05	−0.05	−1	−0.05
Row 3	+1	Wall	−0.05	−0.05	−0.05	+1	−0.05	−0.05	−0.05	Wall
Row 4	−0.05	−0.05	−0.05	Start	Wall	−1	−1	−1	−0.05	−0.05
Row 5	−0.05	−0.05	−0.05	−1	−0.05	−0.05	Wall	−0.05	+1	+1
Row 6	−1	−0.05	−0.05	−0.05	−0.05	Wall	Wall	−0.05	Wall	−0.05
Row 7	−0.05	Wall	−1	−0.05	Wall	−0.05	−0.05	−1	−0.05	−0.05
Row 8	−0.05	−1	−0.05	−0.05	−0.05	−0.05	−0.05	−0.05	−0.05	+1
Row 9	−0.05	−0.05	−0.05	−0.05	−0.05	Wall	−0.05	−0.05	−0.05	−0.05

Table 7: Convergence comparison for the Increased-Size maze.

VI iterations: 688 **PI iterations**: 13

Table 8: Optimal policy for the Increased-Size maze.

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	↑	←	←	→	↓	↓	↓	↓	→	↑
Row 1	↑	↑	←	▦	↓	←	↓	←	←	↑
Row 2	▦	↑	▦	↓	↓	▦	↓	←	↑	↑
Row 3	↑	▦	↓	←	←	←	←	←	←	▦
Row 4	↑	←	←	←	▦	↑	↑	↑	↑	↓
Row 5	↑	↑	←	←	←	←	▦	→	↓	←
Row 6	↑	↑	↑	←	←	▦	▦	↑	▦	↑
Row 7	↑	▦	↑	↑	▦	↓	←	←	→	↑
Row 8	↑	←	↑	↑	←	←	←	←	→	→
Row 9	↑	←	↑	↑	←	▦	↑	←	↑	↑

Table 9: Converged utilities for the Increased-Size maze.

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	88.27	87.09	85.93	84.00	86.32	86.52	87.04	87.06	87.63	88.76
Row 1	86.04	86.99	85.85	▦	88.94	87.63	88.26	88.16	88.00	87.63
Row 2	▦	85.84	▦	91.37	90.29	▦	88.21	87.24	85.89	86.41
Row 3	99.90	▦	93.99	92.72	91.37	91.16	89.50	87.96	86.58	▦
Row 4	98.41	96.96	95.40	93.80	▦	88.59	87.15	85.75	85.46	85.34
Row 5	96.96	95.69	94.40	92.14	90.90	89.58	▦	85.35	86.58	86.46
Row 6	94.49	94.31	93.14	91.81	90.63	▦	▦	84.22	▦	85.32
Row 7	93.24	▦	90.72	90.61	▦	85.40	84.38	82.46	82.97	84.06
Row 8	91.72	89.31	89.48	89.28	87.98	86.67	85.26	83.85	83.15	84.40
Row 9	90.37	89.19	88.36	88.02	86.98	▦	84.01	83.00	82.25	83.17

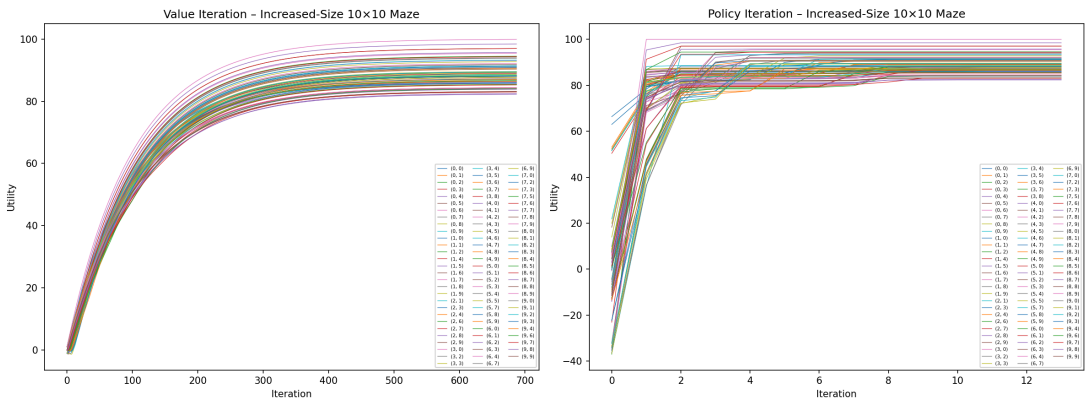

Figure 6: All-states utility convergence for the Increased-Size 10×10 maze.

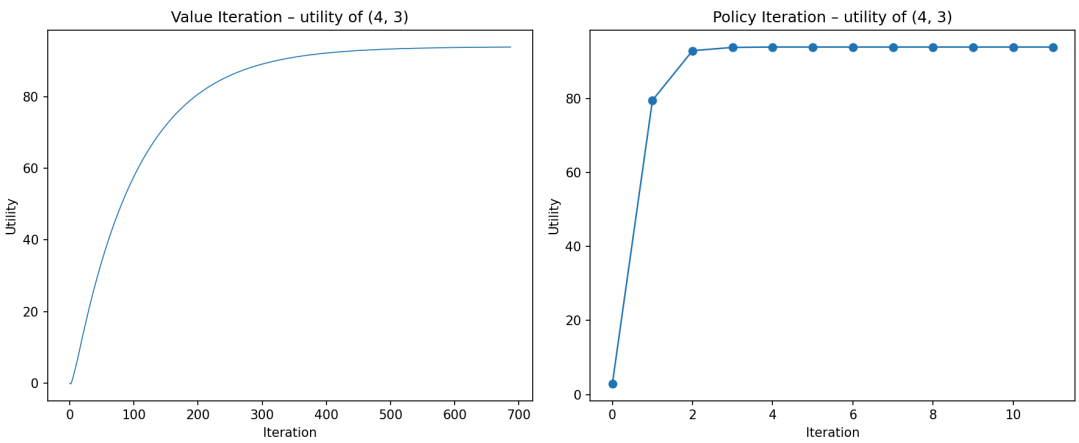

Figure 7: Tracked-state (4, 3) convergence for the Increased-Size 10×10 maze.

Analysis: Despite nearly tripling the state space, Value Iteration still requires exactly 688 sweeps to reach its tolerance threshold, matching the base maze. This visually reinforces the principle that VI's convergence horizon is dictated by the $\gamma = 0.99$ discount factor, not volumetric grid size. Policy Iteration, however, exhibits sensitivity

to the expanded geometry. Because the environment is mostly open, almost every state possesses four viable actions. This high branching factor, combined with the scattered utility gradient, requires PI to undergo 13 distinct policy improvement steps, nearly double the requirement of the base maze, to successfully lock in the optimal sequence of actions.

4.2 Maze 2: Labyrinth (10×10, 67 states, 33 walls)

This maze features a high wall density that forms narrow, winding corridors. Despite having fewer traversable states (67) than the increased-size maze, the physical structure forces the agent into highly constrained, prolonged paths.

Table 10: Labyrinth 10×10 maze layout. Green = +1, Brown = −1, White = −0.05. Start at (5, 3).

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**	**Col 6**	**Col 7**	**Col 8**	**Col 9**
Row 0	−0.05	Wall	**+1**	−0.05	Wall	−0.05	−0.05	Wall	−0.05	**+1**
Row 1	−0.05	Wall	−0.05	**−1**	−0.05	Wall	−0.05	−0.05	Wall	−0.05
Row 2	−0.05	−0.05	−0.05	Wall	−0.05	Wall	−0.05	−0.05	Wall	**+1**
Row 3	**+1**	Wall	−0.05	Wall	−0.05	−0.05	Wall	−0.05	Wall	−0.05
Row 4	Wall	−0.05	−0.05	Wall	−0.05	Wall	−0.05	−0.05	Wall	−0.05
Row 5	−0.05	−0.05	Wall	Start	**+1**	Wall	−0.05	Wall	−0.05	−0.05
Row 6	Wall	−0.05	−0.05	−0.05	Wall	−0.05	−0.05	Wall	−0.05	**+1**
Row 7	−0.05	−0.05	Wall	−0.05	Wall	−0.05	Wall	−0.05	−0.05	Wall
Row 8	−0.05	Wall	−0.05	−0.05	−0.05	**+1**	Wall	−0.05	Wall	−0.05
Row 9	−0.05	−0.05	−0.05	Wall	−0.05	Wall	−0.05	−0.05	Wall	**+1**

Table 11: Optimal policy for the Labyrinth maze.

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**	**Col 6**	**Col 7**	**Col 8**	**Col 9**
Row 0	↓	▦	←	←	▦	→	↓	▦	→	↑
Row 1	↓	▦	↓	←	←	▦	↓	↓	▦	↑
Row 2	↓	←	←	▦	↑	▦	→	↓	▦	↑
Row 3	↓	▦	↑	▦	↑	←	▦	↓	▦	↑
Row 4	▦	→	↑	▦	↓	▦	↓	←	▦	↑
Row 5	→	↑	▦	→	→	▦	↓	▦	→	↓
Row 6	▦	↑	←	←	▦	↓	←	▦	→	→
Row 7	→	↑	▦	↑	▦	↓	▦	→	↑	▦
Row 8	↑	▦	→	→	→	→	▦	↑	▦	↓
Row 9	↑	←	↑	▦	↑	▦	→	↑	▦	↓

Table 12: Converged utilities for the Labyrinth maze.

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	95.87	▦	94.52	92.04	▦	70.93	71.89	▦	87.21	88.38
Row 1	97.14	▦	93.97	91.61	90.11	▦	74.18	75.07	▦	87.21
Row 2	98.43	97.14	95.52	▦	87.74	▦	75.07	76.19	▦	87.37
Row 3	99.90	▦	94.27	▦	85.26	84.13	▦	77.36	▦	86.22
Row 4	▦	91.47	92.86	▦	87.21	▦	80.78	78.40	▦	85.08
Row 5	88.93	90.12	▦	87.00	88.38	▦	82.16	▦	84.92	85.96
Row 6	▦	88.79	87.61	85.34	▦	85.88	84.47	▦	85.69	87.24
Row 7	86.16	87.45	▦	84.21	▦	87.21	▦	81.99	83.22	▦
Row 8	85.02	▦	83.43	84.68	87.07	88.38	▦	80.90	▦	98.59
Row 9	83.76	82.65	82.36	▦	85.92	▦	77.48	78.52	▦	99.90

Table 13: Convergence comparison for the Labyrinth maze.

VI iterations: 688 **PI iterations**: 8

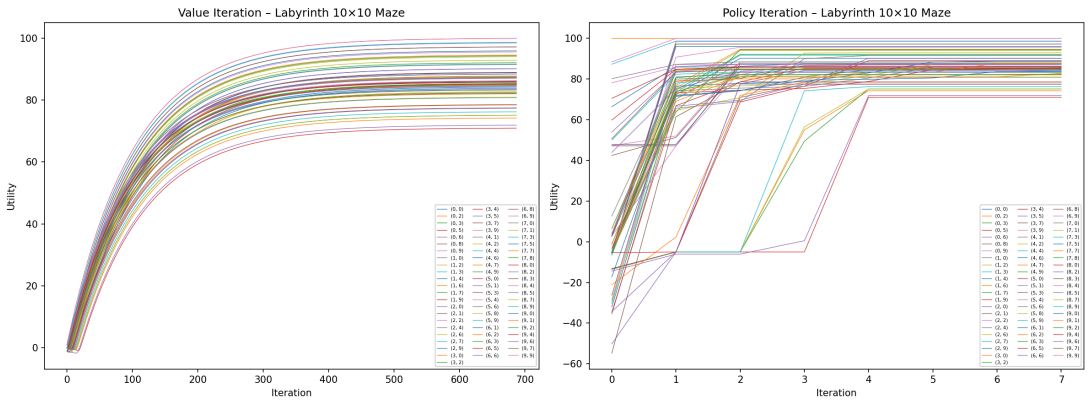

Figure 8: All-states utility convergence for the Labyrinth 10×10 maze.

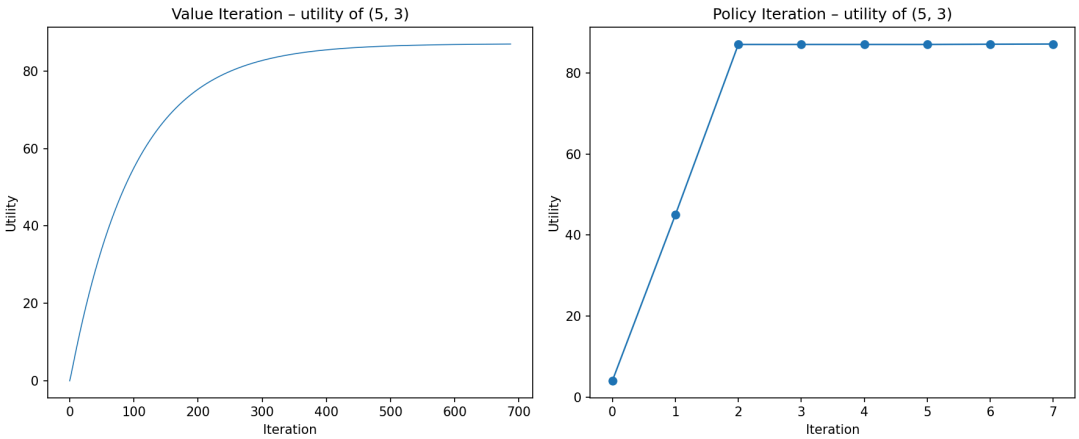

Figure 9: Tracked-state (5, 3) convergence for the Labyrinth 10×10 maze.

Analysis: The Labyrinth environment provides a stark contrast to the open expansion grid. By introducing 33 walls, the maze physically restricts the effective action space; an agent caught in a 1-cell-wide corridor practically has only two viable directional choices rather than four. As a result, Policy Iteration converges in a highly efficient 8 iterations. The tight structural constraints inherently limit the number of possible policy variations, meaning the exact linear evaluation snaps to the optimal corridor routing almost immediately. Value Iteration remains anchored at 688 iterations, proving its total indifference to structural density.

4.3 Maze 3: Minefield (10×10, 90 states, 10 walls)

This environment relies on a dense scattering of −1 penalty traps with minimal wall protection. It tests the algorithms' ability to balance the accumulation of −0.05 step costs against the 20% probabilistic risk of sliding laterally into severe penalties.

Table 14: Minefield 10×10 maze layout. Green = +1, Brown = −1, White = −0.05. Start at (9, 4).

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	−0.05	−0.05	−0.05	−0.05	+1	+1	−0.05	−0.05	−0.05	−0.05
Row 1	−0.05	Wall	−1	−0.05	−1	−1	−0.05	−1	Wall	−0.05
Row 2	−0.05	−1	−0.05	−0.05	−0.05	−0.05	−0.05	−0.05	−1	−0.05
Row 3	−0.05	−0.05	−0.05	Wall	−1	−1	Wall	−0.05	−0.05	−0.05
Row 4	−1	Wall	−0.05	−1	−0.05	−0.05	−1	−0.05	Wall	−1
Row 5	−0.05	−1	−0.05	−0.05	−0.05	−0.05	−0.05	−0.05	−1	−0.05
Row 6	−0.05	−0.05	−0.05	Wall	−1	−1	Wall	−0.05	−0.05	−0.05
Row 7	−0.05	−1	−0.05	−0.05	−1	−1	−0.05	−0.05	−1	−0.05
Row 8	−0.05	Wall	−1	−0.05	−0.05	−0.05	−0.05	−1	Wall	−0.05
Row 9	−0.05	−0.05	−0.05	−0.05	Start	−0.05	−0.05	−0.05	−0.05	−0.05

Table 15: Optimal policy for the Minefield maze.

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	→	→	→	→	↑	↑	←	←	←	←
Row 1	↑	▦	↑	↑	↑	↑	↑	↑	▦	↑
Row 2	↑	→	→	↑	↑	↑	↑	←	←	↑
Row 3	↑	→	↑	▦	↑	↑	▦	↑	←	↑
Row 4	↑	▦	↑	→	↑	↑	←	↑	▦	↑
Row 5	↑	→	↑	→	↑	↑	←	↑	←	↑
Row 6	↑	→	↑	▦	↑	↑	▦	↑	←	↑
Row 7	↑	↑	↑	←	↑	↑	→	↑	↑	↑
Row 8	↑	▦	↑	↑	↑	↑	↑	↑	▦	↑
Row 9	↑	→	↑	↑	↑	↑	↑	↑	←	↑

Table 16: Converged utilities for the Minefield maze.

	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
Row 0	81.85	83.08	84.20	85.58	86.89	86.89	85.58	84.20	83.08	81.85
Row 1	80.77	▦	82.16	84.23	84.53	84.53	84.23	82.16	▦	80.77
Row 2	79.68	79.54	81.81	83.01	83.37	83.37	83.01	81.81	79.54	79.68
Row 3	78.72	79.52	80.59	▦	81.08	81.08	▦	80.59	79.52	78.72
Row 4	76.49	▦	79.30	77.49	79.73	79.73	77.49	79.30	▦	76.49
Row 5	75.50	75.68	77.92	77.50	78.54	78.54	77.50	77.92	75.68	75.50
Row 6	74.60	75.49	76.73	▦	76.31	76.31	▦	76.73	75.49	74.60
Row 7	73.60	73.53	75.35	74.22	74.12	74.12	74.22	75.35	73.53	73.60
Row 8	72.62	▦	73.16	73.22	73.15	73.15	73.22	73.16	▦	72.62
Row 9	71.59	71.12	72.09	72.22	72.18	72.18	72.22	72.09	71.12	71.59

Table 17: Convergence comparison for the Minefield maze.

VI iterations: 674 **PI iterations**: 8

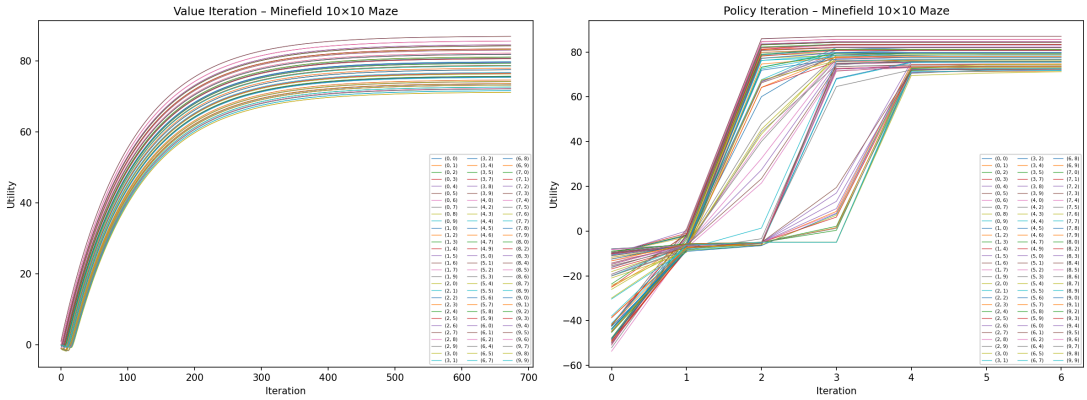

Figure 10: All-states utility convergence for the Minefield 10×10 maze.

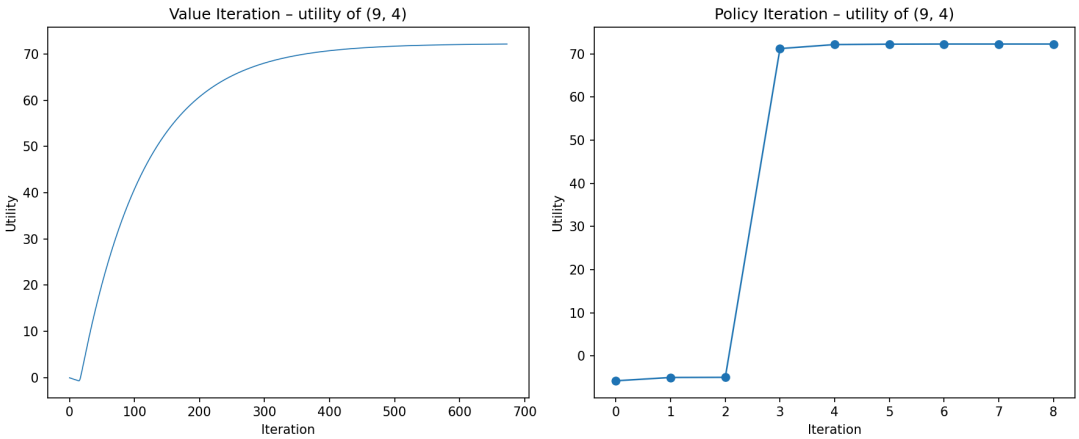

Figure 11: Tracked-state (9, 4) convergence for the Minefield 10×10 maze.

Analysis: The Minefield isolates the tension of stochastic risk tolerance. Because the agent has a 10% chance to slip left and a 10% chance to slip right during any intended movement, navigating through adjacent penalties carries a massive mathematical risk. The resulting optimal policy beautifully visualises the Bellman equation's risk aversion, where the agent actively routes itself into distinct vertical "safe columns" (such as columns 4 and 5) where upward movement is shielded by walls or void of adjacent traps, along with boundary escape corridors (such as columns 0 and 9). Value Iteration converges slightly faster here at 674 iterations, a byproduct of the heavy penalty density artificially depressing the infinite horizon maximum utilities (topping out at ~87 instead of ~100), thereby satisfying the tolerance threshold slightly earlier.

4.4 Analysis: Effect of Complexity on Convergence

Table 18: Convergence summary across all four environments.

Maze	Grid	States	Walls	VI Iters	PI Iters
Base	6×6	31	5	688	6–8
Increased-Size	10×10	86	14	688	13
Labyrinth	10×10	67	33	688	8
Minefield	10×10	90	10	674	8

4.4.1 How does the number of states and the complexity of the environment affect convergence?

Value Iteration convergence is overwhelmingly dominated by the discount factor, rather than geometric complexity. With $\gamma = 0.99$, the AIMA tolerance equation establishes an incredibly strict threshold. Consequently, VI demands 670 to 690 sweeps regardless of whether the grid contains 31 states or 90, or whether those states are open fields or constrained corridors. Because VI's convergence rate scales functionally as $O(1/(1-\gamma))$, physical maze alterations only marginally affect the finish line.

Policy Iteration is highly sensitive to action-space freedom. The open Increased-Size maze (86 states, only 14 walls) demanded the most PI iterations (13) because almost every state offered four unconstrained directional choices, creating a vast combinatorial policy space. In stark contrast, the highly constrained Labyrinth (67 states, 33 walls) restricted available actions to narrow corridors, shrinking the policy space and allowing PI to solve the exact linear system in just 8 steps.

4.4.2 How complex can you make the environment and still be able to learn the right policy?

Both algorithms flawlessly deduced the mathematically optimal policies across all environmental configurations up to 90 states. For environments at this 10×10 scale, the exact Bellman matrix inversion used in Policy Iteration ($O(|S|^3)$) occurs in a fraction of a second, making it significantly more efficient than Value Iteration's 688 sweeping approximations. However, as MDP complexity scales towards thousands of states (e.g. a 50×50 grid), the cubic cost of PI's exact linear solve will eventually hit a computational bottleneck. At highly extended complexities, the field relies on Modified Policy Iteration (using iterative approximations instead of exact solves) or Deep Reinforcement Learning (function approximation) to maintain tractability.

Chapter 5

Conclusion

Both Value Iteration and Policy Iteration successfully and consistently computed the optimal policies for the stochastic maze MDP. Across all tested environments, from the foundational 6×6 grid to the complex 10×10 variations, both algorithms produced identical behavioural mappings, confirming the mathematical correctness of their implementations. However, their pathways to convergence fundamentally differ. Policy Iteration leverages exact linear algebra to evaluate policies in a single mathematical step, allowing it to converge and stabilise from a mere 6 to 13 iterations. Conversely, Value Iteration relies on the asymptotic propagation of value information across the grid, predictably demanding roughly 674 to 688 sweeping updates to satisfy the strict tolerance threshold established by the high discount factor ($\gamma = 0.99$).

The experiments conducted in Part 2 successfully isolated the distinct environmental factors that influence algorithmic performance. Volumetric expansion (the Increased-Size maze) demonstrated that drastically scaling an open state space inflates the combinatorial policy space, requiring more iteration cycles from Policy Iteration, while Value Iteration's sweep count remained stubbornly anchored to the discount factor. Introducing severe structural density (the Labyrinth maze) revealed an inverse relationship for Policy Iteration, heavily constrained corridors artificially shrink the effective action space, actively reducing the number of policy improvement steps required to snap to the optimal route. Finally, the introduction of dense hazard placements (the Minefield maze) proved that both algorithms flawlessly handle stochastic risk tolerance, gracefully balancing the slow accumulation of step costs against the severe mathematical probability of slipping into penalty traps.

Ultimately, the choice between these foundational algorithms hinges almost entirely on environmental scale. Policy Iteration operates incredibly efficiently when the exact inversion of the transition matrix is computationally trivial, as was the case for the ~100-state grids tested in this report. However, because its evaluation step scales cubically ($O(|S|^3)$), it becomes a computational bottleneck for larger systems. For massive state spaces, Value Iteration's lightweight $O(|S| \times |A|)$ per-sweep up-

dates remain the more viable approach, demonstrating the fundamental scalability tradeoffs that drive modern solutions like Modified Policy Iteration and Deep Reinforcement Learning.